

## PRACTICAL-1

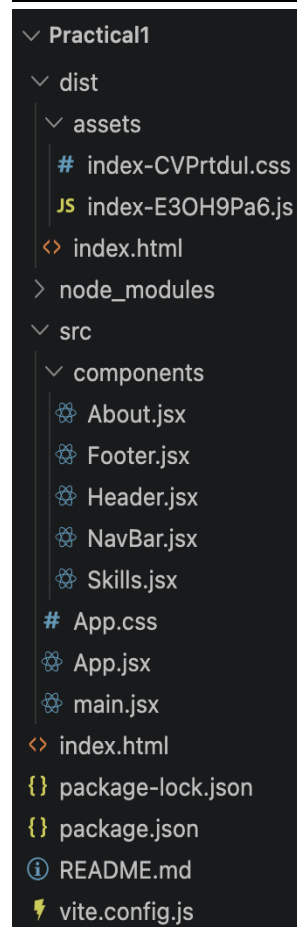
### PROBLEM DEFINITION:

- Create a React application using Vite for a student portfolio page.
- The application must include at least 4 reusable components: Header, About, Skills, and Footer.
- Each component must be independently structured and composed into a single-page layout.
- No logic or JSX should be duplicated across components.
- Props must be used to pass at least one piece of data into a minimum of 2 components.

### SUPPLEMENTARY PROBLEMS:

- Add a NavBar component that highlights the active section.
- Pass an array of skills as a prop to the Skills component and render them dynamically.
- Add a theme color prop to the Header and apply it as an inline style.

### ARCHITECTURE / DIAGRAM:



**CODE (SUPPLEMENTARY PROBLEMS INCLUDED):****➤ App.jsx**

```
import { useState } from "react";
import Header from "../components/Header";
import NavBar from "../components/NavBar";
import About from "../components/About";
import Skills from "../components/Skills";
import Footer from "../components/Footer";

function App()
{
  const [activeSection, setActiveSection] = useState("about");

  const skills = ["HTML", "CSS", "JavaScript", "React", "C++", "Python", "MongoDB"];

  const student = {
    name: "Jainam Kamani",
    bio: "I am an Information Technology student interested in web development, software
    development, and data science.",
  };

  return (
    <div className="app">
      <Header name={student.name} themeColor="#2563eb" />

      <NavBar
        activeSection={activeSection}
        setActiveSection={setActiveSection}
      />

      <main>
        <About
          name={student.name}
          bio={student.bio}
        />

        <Skills skills={skills} />
      </main>

      <Footer email="jainamkamani95@gmail.com" />
    </div>
```

```
);  
}
```

```
export default App;
```

➤ **Header.jsx**

```
function Header({ name, themeColor })  
{  
  return (  
    <header className="header" style={{ backgroundColor: themeColor }}>  
      <h1>Student Portfolio</h1>  
      <p>{name}</p>  
    </header>  
  );  
}
```

```
export default Header;
```

```
function About({ name, bio })  
{  
  return (  
    <section id="about" className="section about">  
      <h2>About Me</h2>  
      <p>Hello, I am <strong>{name}</strong>.</p>  
      <p>{bio}</p>  
    </section>  
  );  
}
```

```
export default About;
```

➤ **Footer.jsx**

```
function Footer({ email })  
{  
  return (  
    <footer className="footer">  
      <p>Contact: {email}</p>  
      <p>© 2026 Jainam Kamani. All rights reserved.</p>  
    </footer>  
  );  
}
```

```
}
```

```
export default Footer;
```

➤ **NavBar.jsx**

```
function NavBar({ activeSection, setActiveSection })  
{  
  const sections = ["about", "skills"];
```

```
  const handleClick = (section) => {  
    setActiveSection(section);  
    document.getElementById(section)?.scrollIntoView({  
      behavior: "smooth",  
    });  
  };  
};
```

```
  return (  
    <nav className="navbar">  
      {sections.map((section) => (  
        <button  
          key={section}  
          className={activeSection === section ? "active" : ""}  
          onClick={() => handleClick(section)}  
        >  
          {section.charAt(0).toUpperCase() + section.slice(1)}  
        </button>  
      ))}  
    </nav>  
  );  
}
```

```
export default NavBar;
```

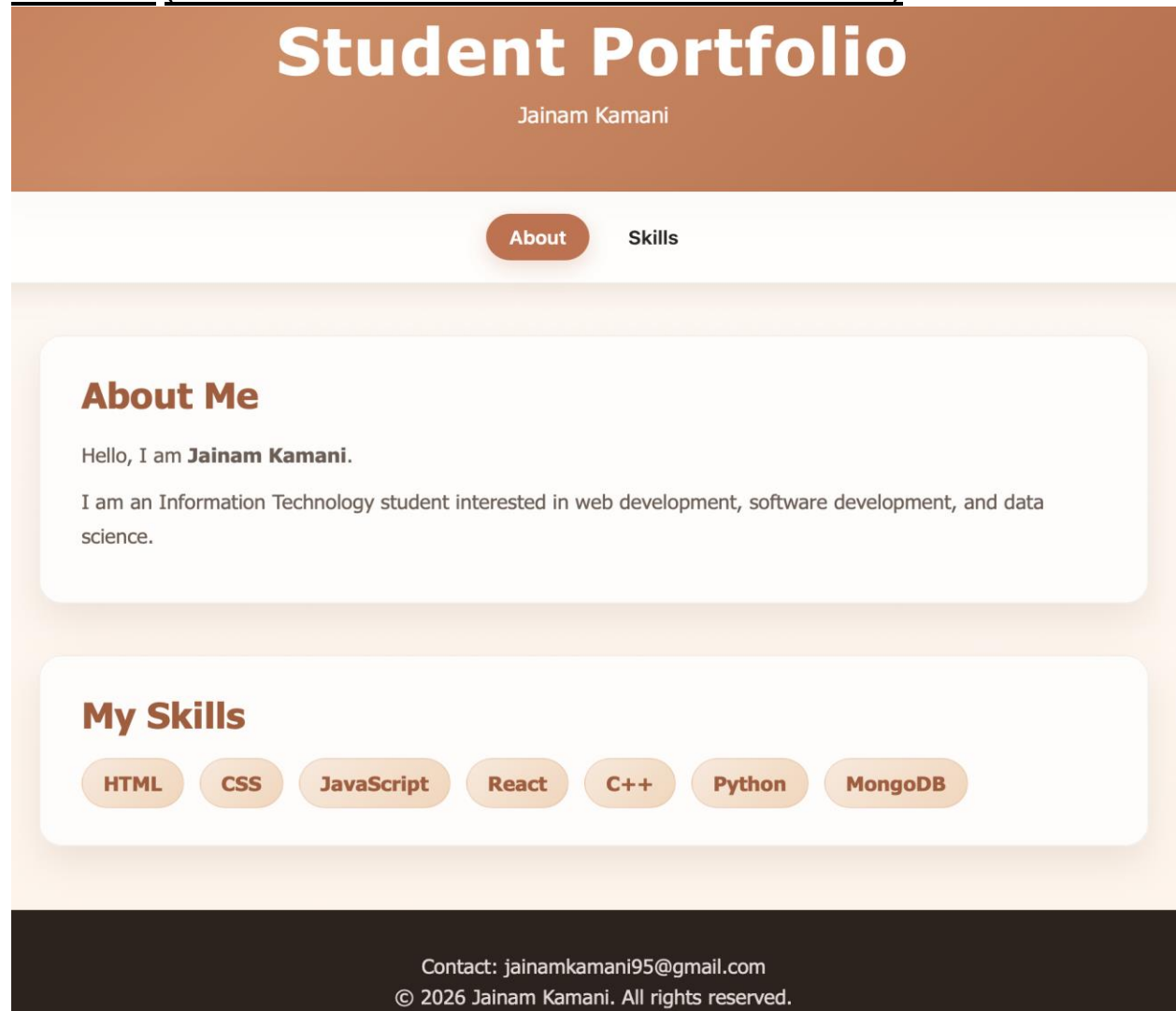
➤ **Skills.jsx**

```
function Skills({ skills })  
{  
  return (  
    <section id="skills" className="section">  
      <h2>My Skills</h2>
```

```
<div className="skills-list">
  {skills.map((skill) => (
    <span className="skill" key={skill}>
      {skill}
    </span>
  ))}
</div>
</section>
);
}
```

```
export default Skills;
```

### OUTPUT (SUPPLEMENTARY PROBLEMS INCLUDED):



**KEY QUESTIONS / ANALYSIS:****1) What is the role of each component in the overall UI structure?**

- Header: Displays the student's name, profile title, and navigation at the top of the portfolio.
- About: Shows a brief introduction, educational background, and career objective of the student.
- Skills: Displays the student's technical and soft skills, which can be passed through props for reusability.
- Footer: Displays contact information, copyright details, or social media links at the bottom of the page.

These components work together to create a well-structured, single-page student portfolio while keeping the code modular and easy to maintain.

**2) How does React re-render when props change?**

In this portfolio application, data such as the student's name or skills is passed from the parent component (App.jsx) to child components using props. Whenever the parent sends updated props, React automatically re-renders only the affected components. React uses the Virtual DOM to compare the previous and updated UI and updates only the changed elements in the browser, making rendering fast and efficient.

**3) Why is component reusability important in large-scale applications?**

Component reusability allows the same component to be used multiple times without rewriting code. In this portfolio application, components like Header, Skills, About, and Footer are independent and reusable. Data can be changed using props without modifying the component itself. This reduces code duplication, improves consistency, simplifies maintenance, and makes large-scale React applications easier to develop and manage.

**GITHUB REPOSITORY:**

<https://github.com/9081495207/ITUE301-Advanced-Web-Development-Frameworks>